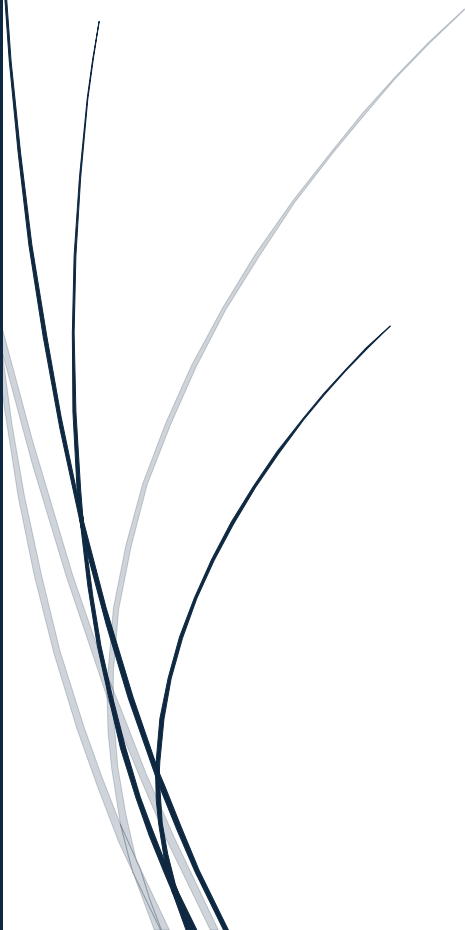


21/10/2025

Esercizio Shell – Gestione Directory e Files

A cura di Fabio Renna



Titolo: Esercizi Shell – Creazione e Gestione Directory e File

Autore: Fabio Renna

Data: 21/10/2025

Ambiente di lavoro: Kali Linux

Indice:

1. Executive Summary
2. Obiettivi dell'esercizio
3. Proof of Concept (PoC)
4. Considerazioni finali
5. Conclusione

1. Executive Summary

In questo esercizio ho utilizzato i principali comandi della shell Linux per creare, navigare e gestire directory e files. Partendo dalla directory *HOME*, ho costruito una struttura ad albero con più sottocartelle e file, quindi ho eseguito operazioni di copia, spostamento, rimozione, modifica dei permessi e visualizzazione dei contenuti. L'obiettivo era acquisire familiarità con i comandi base del terminale, comprendendo il funzionamento dei percorsi assoluti e relativi e la gestione dei file system.

2. Obiettivi dell'esercizio

- Creare e gestire directory e sottodirectory.
- Usare i percorsi assoluti e relativi.
- Copiare, spostare e cancellare file e cartelle.
- Gestire i permessi dei file.
- Visualizzare contenuti e percorsi tramite comandi base.

3. Proof of Concept (PoC)

L'esercizio di oggi parte con la creazione dalla Home di 4 Directory che, a loro volta, avranno altre sottocartelle.

Le cartelle da creare sono: “**dos**” – “**studenti**” – “**windows**” – “**tmp**” che verranno create tramite il comando “**mkdir**” (Make Directory) come da esempio in foto:

```
(kali@kali)-[~]
$ pwd
/home/kali
(kali@kali)-[~]
$ mkdir dos && mkdir studenti && mkdir windows && mkdir tmp
(kali@kali)-[~]
$ ls
Desktop  Documents  dos  Downloads  exit  force  install  Music  Pictures  Public  studenti  Templates  'TEST DI PROVA'  tmp  Videos  windows
```

Figura 1 Dopo essermi accertato di essere nella Home tramite il comando “**pwd**” ho creato le 4 cartelle richieste per poi verificarle con “**ls**”

Ora l’esercizio chiede di creare un file all’interno della cartella “**tmp**”, nominato: “**risultati.doc**” che sono andato a creare tramite un Editor di testo da terminale molto utilizzato in Linux: “**nano**”:

```
(kali@kali)-[~]
$ ls
Desktop  Documents  dos  Downloads  exit  force  install  Music  Pictures  Public  studenti  Templates  'TEST DI PROVA'  tmp  Videos  windows
(kali@kali)-[~]
$ cd tmp
(kali@kali)-[~/tmp]
$ nano risultati.doc
```

```
Session  Actions  Edit  View  Help
GNU nano 8.6
file di prova "risultati.doc" per esercizio a cura di Fabio Renna
```

```
(kali@kali)-[~]
$ ls
Desktop  Documents  dos  Downloads  exit  force  install  Music  Pictures  Public  studenti  Templates  'TEST DI PROVA'  tmp  Videos  windows
(kali@kali)-[~]
$ cd tmp
(kali@kali)-[~/tmp]
$ nano risultati.doc
(kali@kali)-[~/tmp]
$ ls
risultati.doc
(kali@kali)-[~/tmp]
$
```

Figura 2 Dopo aver scritto e salvato il file, mi sono accertato della corretta riuscita tramite il comando “**ls**”.

Successivamente, l’esercizio ci chiedeva di creare delle sottocartelle all’interno di “**studenti**” nominate: “**nicola**” – “**anna**” – “**matteo**”.

```

(kali㉿kali)-[~]
$ cd studenti

(kali㉿kali)-[~/studenti]
$ mkdir matteo && mkdir anna && mkdir nicola

(kali㉿kali)-[~/studenti]
$ ls
anna matteo nicola

```

Figura 3 Dopo essere entrato nella cartella "**studenti**" con il comando "**cd**" (change directory), ho creato tramite il comando "**mkdir**" le cartelle richieste

L'esercizio poi ci chiede di creare la cartella "**amici**" all'interno della cartella "**matteo**",

```

(kali㉿kali)-[~/studenti]
$ cd matteo

(kali㉿kali)-[~/studenti/matteo]
$ mkdir amici

(kali㉿kali)-[~/studenti/matteo]
$ ls
amici

```

La cartella "**casa**" all'interno della cartella "**anna**",

```

(kali㉿kali)-[~/studenti/matteo]
$ cd ..

(kali㉿kali)-[~/studenti]
$ cd anna

(kali㉿kali)-[~/studenti/anna]
$ mkdir casa

(kali㉿kali)-[~/studenti/anna]
$ ls
casa

```

E 2 cartelle all'interno di "**nicola**", nominate: "**scuola**" e "**lavoro**":

```

(kali㉿kali)-[~/studenti/anna]
$ cd ..

(kali㉿kali)-[~/studenti]
$ cd nicola

(kali㉿kali)-[~/studenti/nicola]
$ mkdir scuola && mkdir lavoro

(kali㉿kali)-[~/studenti/nicola]
$ ls
lavoro  scuola

```

Infine, all'interno della cartella “**scuola**” ci chiede di creare 2 files, chiamati: “**relazione.doc**” e “**compito.doc**” che andremo a creare tramite il comando “**nano**” visto già in precedenza:

```

(kali㉿kali)-[~/studenti/nicola/scuola]
$ nano compito.doc

```

```

Session  Actions  Edit  View  Help
GNU nano 8.6
file di prova per esercizio a cura di Fabio Renna

```

```

(kali㉿kali)-[~/studenti/nicola/scuola]
$ nano relazione.doc

```

```

Session  Actions  Edit  View  Help
GNU nano 8.6
file di prova per esercizio "relazione.doc" a cura di Fabio Renna

```

```

(kali㉿kali)-[~]
$ ls
Desktop  Documents  dos  Downloads  exit  force  install  Music  Pictures  Public  studenti  Templates  'TEST DI PROVA'  tmp  Videos  windows

(kali㉿kali)-[~]
$ cd tmp

(kali㉿kali)-[~/tmp]
$ nano risultati.doc

(kali㉿kali)-[~/tmp]
$ ls
risultati.doc

(kali㉿kali)-[~/tmp]
$

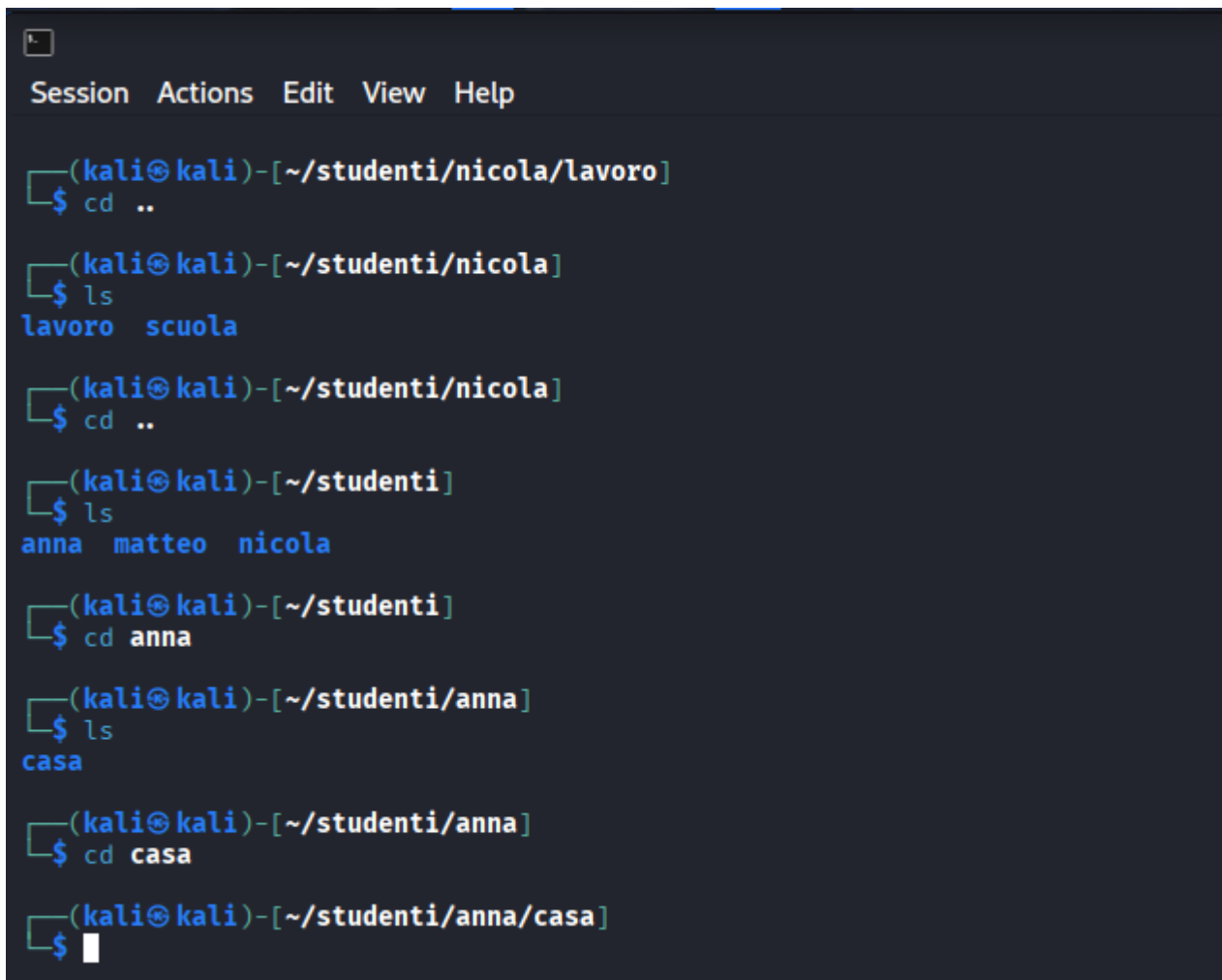
```

Figura 4 Dopo la creazione dei file di testo siamo andati a verificare il corretto salvataggio tramite il comando “**ls**”

L'esercizio continua chiedendoci di spostarci dalla directory “**lavoro**” (sotto “**nicola**”) alla directory “**anna**” con il percorso relativo e percorso assoluto.

La differenza sostanziale tra i due tipi di percorsi è il punto di partenza:

- Mentre il percorso **relativo** indica la posizione del file o directory in base alla posizione corrente, usando scorciatoie come "**cd <nome directory>**" per andare avanti e "**cd ..**" per tornare indietro nel percorso:



```
Session Actions Edit View Help

(kali㉿kali)-[~/studenti/nicola/lavoro]
$ cd ..

(kali㉿kali)-[~/studenti/nicola]
$ ls
lavoro  scuola

(kali㉿kali)-[~/studenti/nicola]
$ cd ..

(kali㉿kali)-[~/studenti]
$ ls
anna  matteo  nicola

(kali㉿kali)-[~/studenti]
$ cd anna

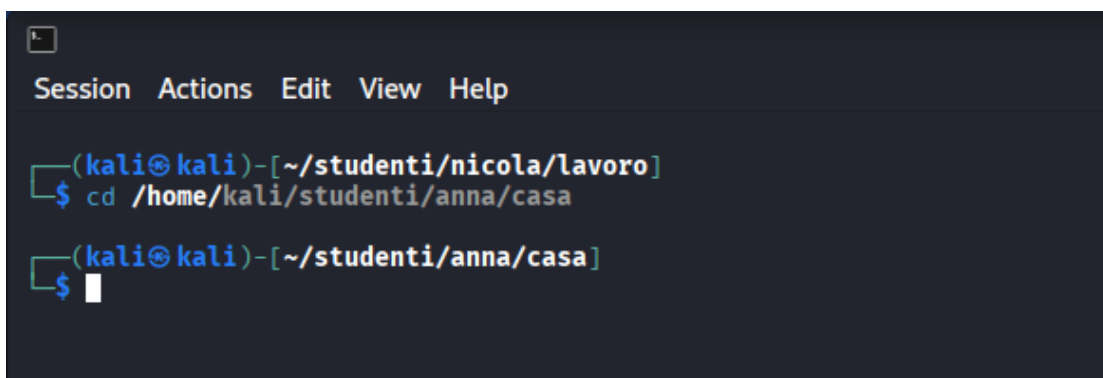
(kali㉿kali)-[~/studenti/anna]
$ ls
casa

(kali㉿kali)-[~/studenti/anna]
$ cd casa

(kali㉿kali)-[~/studenti/anna/casa]
$
```

Figura 5 Percorso relativo per raggiungere "casa" dalla directory "lavoro" spostandosi con i comandi "cd .." - "ls" - "cd"

- Il percorso **assoluto** indica la posizione del file o della directory partendo dalla **radice**, usando come stringa di comando l'intero percorso:



```
Session Actions Edit View Help

(kali㉿kali)-[~/studenti/nicola/lavoro]
$ cd /home/kali/studenti/anna/casa

(kali㉿kali)-[~/studenti/anna/casa]
$
```

Ora l'esercizio ci chiede di destreggiarci con la gestione dei files e directory con comandi per copiarle, spostarle, eliminarle e perfino cambiare autorizzazioni.

Andiamo con ordine, iniziando da copiare il file “**compito.doc**” precedentemente creato nella directory “**scuola**”, all’interno directory corrente “**casa**”.

Non ci resta che utilizzare il comando “**cp**” (copy) <percorso del file “**compito.doc**”> e “**.**” dall’interno della directory corrente “**casa**” in modo da indirizzare il file all’interno della stessa:

```
(kali@kali)-[~/studenti/anna/casa]
$ cp /home/kali/studenti/nicola/scuola/compito.doc .
(kali@kali)-[~/studenti/anna/casa]
$ ls
compito.doc  relazione.doc
(kali@kali)-[~/studenti/anna/casa]
$ cat compito.doc
file di prova per esercizio a cura di Fabio Renna
(kali@kali)-[~/studenti/anna/casa]
$
```

Ora, invece, dobbiamo spostare il file “**relazione.doc**” dalla directory “**scuola**” alla directory “**casa**” corrente, ed utilizziamo lo stesso metodo, a differenza del comando iniziale, che invece di essere “**cp**”, sarà “**mv**” (move).

La differenza è che questo comando non genererà una copia del file, bensì un vero e proprio spostamento “eliminando” traccia dalla sua vecchia directory.

```
Session  Actions  Edit  View  Help
(kali@kali)-[~/studenti/anna/casa]
$ mv /home/kali/studenti/nicola/scuola/relazione.doc .
(kali@kali)-[~/studenti/anna/casa]
$ ls
compito.doc  relazione.doc
(kali@kali)-[~/studenti/anna/casa]
$ cat relazione.doc
file di prova per esercizio "relazione.doc" a cura di Fabio Renna
(kali@kali)-[~/studenti/anna/casa]
$
```

Procediamo ora con l'eliminazione della directory **"tmp"**, contenente al suo interno altri file o sottocartelle. Dopo essere tornati nella **home** tentiamo inizialmente di rimuoverla con il comando **"rmdir -r"** <nome directory>.

Tuttavia, questo genera un errore, poiché il comando **rmdir** può eliminare **solo directory vuote**. Per eliminare correttamente una directory piena, è necessario utilizzare invece **"rm -r"** <nome directory>.

Il parametro **"-r"** indica di cancellare tutti i file e sottodirectory presenti.

```
Session Actions Edit View Help
(kali@kali)-[~]
$ ls
Desktop Documents dos Downloads exit force install Music Pictures Public studenti Templates 'TEST DI PROVA' tmp Videos windows
(kali@kali)-[~]
$ rmdir -r tmp
rmdir: invalid option -- 'r'
Try 'rmdir --help' for more information.
(kali@kali)-[~]
$ rm -r tmp
(kali@kali)-[~]
$ ls
Desktop Documents dos Downloads exit force install Music Pictures Public studenti Templates 'TEST DI PROVA' Videos windows
(kali@kali)-[~]
$
```

Il prossimo passo è creare il file **"pippo.txt"** all'interno della cartella **"lavoro"**.

Ci spostiamo, dunque, all'interno della directory **"lavoro"** ed utilizziamo lo stesso metodo precedente per creare un file di testo: l'editor **"nano"**:

```
(kali@kali)-[~]
$ cd /home/kali/studenti/nicola/lavoro
(kali@kali)-[~/studenti/nicola/lavoro]
$ nano pippo.txt
```

```
Session Actions Edit View Help
GNU nano 8.6
file di prove "pippo.txt" per esercizio a cura di Fabio Renna
```

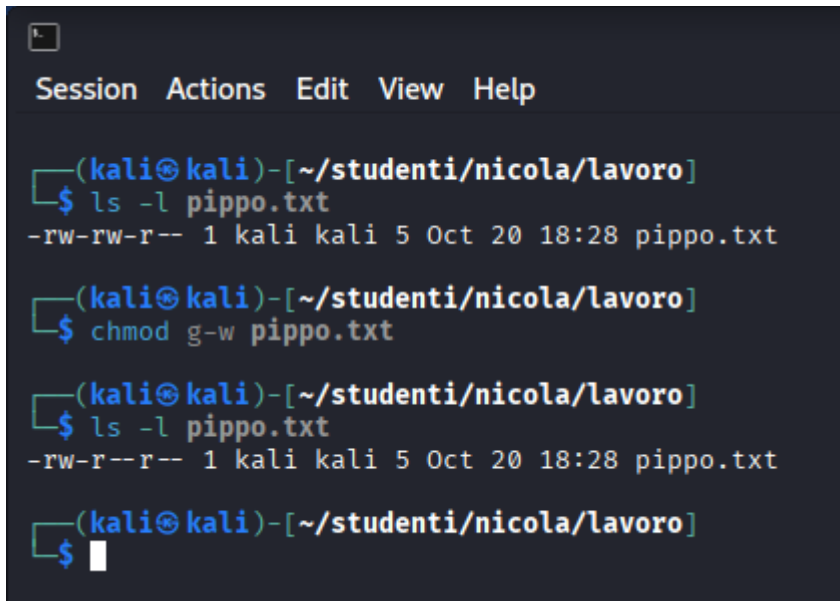
```
(kali@kali)-[~]
$ cd /home/kali/studenti/nicola/lavoro
(kali@kali)-[~/studenti/nicola/lavoro]
$ nano pippo.txt
(kali@kali)-[~/studenti/nicola/lavoro]
$ ls
pippo.txt
(kali@kali)-[~/studenti/nicola/lavoro]
$ cat pippo.txt
file di prove "pippo.txt" per esercizio a cura di Fabio Renna
(kali@kali)-[~/studenti/nicola/lavoro]
$
```


Una volta creato il file “**pippo.txt**”, l’esercizio richiede che solo il **proprietario** possa avere i permessi di lettura e scrittura, mentre **gruppo e altri utenti** devono avere solo il permesso di lettura.

Abbiamo appreso che, per modificare i permessi esistenti, si utilizza il comando “**chmod**”, seguito dalla lettera che indica la classe di utenti: “**u**” (user/proprietario), “**g**” (group/gruppo) oppure “**o**” (others/altri) e dal segno “**+**” per aggiungere permessi o “**-**” per rimuoverli.

I permessi modificabili sono: “**r**” (read/lettura), “**w**” (write/scrittura), “**x**” (execute/esecuzione).

Una volta visti i permessi del file “**pippo.txt**” tramite il comando “**ls -l**”, siamo andati a modificarli come chiesto rimuovendo “**w**” solo alla classe “**g**”, dato che “**o**” non aveva i permessi “**w**” e “**x**” di default.



```
Session Actions Edit View Help

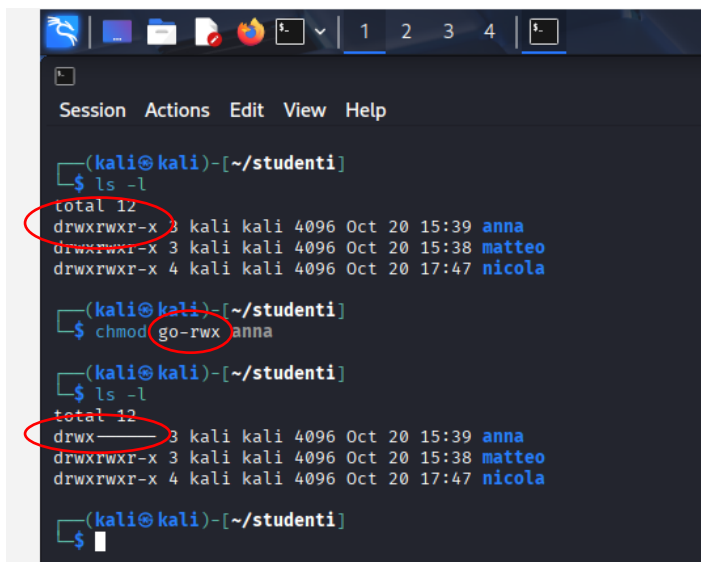
(kali㉿kali)-[~/studenti/nicola/lavoro]
$ ls -l pippo.txt
-rw-rw-r-- 1 kali kali 5 Oct 20 18:28 pippo.txt

(kali㉿kali)-[~/studenti/nicola/lavoro]
$ chmod g-w pippo.txt

(kali㉿kali)-[~/studenti/nicola/lavoro]
$ ls -l pippo.txt
-rw-r--r-- 1 kali kali 5 Oct 20 18:28 pippo.txt

(kali㉿kali)-[~/studenti/nicola/lavoro]
$
```

Adesso, invece, l’esercizio ci chiede di “nascondere” la directory “**anna**” e cioè di renderlo invisibile a tutti (tranne al proprietario). Procediamo quindi ad andare nel percorso dove è situata la directory e con il comando “**ls -l**” per visualizzare i permessi della directory per poi modificarli secondo la traccia, rimuovendo quindi, i permessi “**rwX**” per le classi “**g**” e “**o**”



```
Session Actions Edit View Help

(kali㉿kali)-[~/studenti]
$ ls -l
total 12
drwxrwxr-x 3 kali kali 4096 Oct 20 15:39 anna
drwxrwxr-x 3 kali kali 4096 Oct 20 15:38 matteo
drwxrwxr-x 4 kali kali 4096 Oct 20 17:47 nicola

(kali㉿kali)-[~/studenti]
$ chmod go-rwx anna

(kali㉿kali)-[~/studenti]
$ ls -l
total 12
drwx----- 3 kali kali 4096 Oct 20 15:39 anna
drwxrwxr-x 3 kali kali 4096 Oct 20 15:38 matteo
drwxrwxr-x 4 kali kali 4096 Oct 20 17:47 nicola

(kali㉿kali)-[~/studenti]
$
```

Infine, l'esercizio ci chiede di eliminare prima la directory vuota “**amici**” e, successivamente, tutte le cartelle create per l'esercizio.

Quindi, per quanto riguarda la directory “amici” possiamo utilizzare il comando “**rmdir**” (remove directory) dato che la cartella è vuota:

```
Session Actions Edit View Help

(kali@kali)-[~/studenti/nicola/lavoro]
$ ls
pippo.txt

(kali@kali)-[~/studenti/nicola/lavoro]
$ cd ..

(kali@kali)-[~/studenti/nicola]
$ cd ..

(kali@kali)-[~/studenti]
$ cd matteo

(kali@kali)-[~/studenti/matteo]
$ ls
amici

(kali@kali)-[~/studenti/matteo]
$ rmdir amici

(kali@kali)-[~/studenti/matteo]
$ ls

(kali@kali)-[~/studenti/matteo]
$
```

Mentre per eliminare tutte le cartelle create in precedenza, andiamo ad utilizzare il comando “**rm -r**” dalla nostra home, eliminando le restanti 3 principali directory “**dos**” – “**studenti**” – “**windows**” che contengono tutte le altre subdirectory e files annessi:

```
kali@kali: ~
Session Actions Edit View Help

(kali@kali)-[~]
$ ls
Desktop Documents dos Downloads exit force install Music Pictures Public studenti Templates 'TEST DI PROVA' Videos windows

(kali@kali)-[~]
$ rm -r dos 66 rm -r studenti 66 rm -r windows

(kali@kali)-[~]
$ ls
Desktop Documents Downloads exit force install Music Pictures Public Templates 'TEST DI PROVA' Videos

(kali@kali)-[~]
$
```

4. Considerazioni finali

Durante lo svolgimento dell'esercizio ho avuto modo di mettere in pratica i comandi principali della shell Linux e di capire meglio come muovermi all'interno del file system.

Lavorare con percorsi relativi e assoluti mi ha aiutato a comprendere quanto sia importante prestare attenzione alla posizione corrente e alla sintassi dei comandi.

Ho notato che alcune operazioni, come lo spostamento o la rimozione dei file, richiedono precisione e consapevolezza, perché un errore può facilmente portare alla perdita di dati.

La parte dedicata ai permessi è stata particolarmente utile per capire come Linux gestisce la sicurezza dei file e in che modo i vari utenti possono o non possono accedere alle risorse del sistema.

In generale, l'esercitazione mi ha fatto capire meglio la logica che sta dietro ai comandi del terminale e quanto sia utile conoscerli per lavorare in modo più diretto e controllato sul sistema.

5. Conclusione

Questo esercizio mi ha permesso di prendere confidenza con la shell Linux e di consolidare le basi sull'uso dei comandi più comuni.

Attraverso prove pratiche come la creazione, lo spostamento e la cancellazione di file e cartelle, ho capito quanto sia importante conoscere la struttura delle directory e i diversi tipi di percorso.

Inoltre, imparare a gestire i permessi con `chmod` mi ha dato una visione più chiara del controllo che Linux offre sui file e sulla sicurezza del sistema.

Nel complesso, è stata un'attività utile per aumentare la mia autonomia nel terminale e per prepararmi a esercizi più avanzati nel percorso di cybersecurity.

Venezia, 21/10/2025

Fabio Renna